

DNF 刷图代码逐段讲解版

清爽修正版：修复乱码、去掉大量空白分页、保留“上面代码，下面解释”的排版方式。

这版我重新整理过两件事：第一，代码片段统一使用当前工作区里已经正常的中文源码；第二，去掉封面大留白和章节强制换页，改成连续阅读排版。你可以直接从上往下读，不会再出现大片空白。

当前真正参与刷图主流程的文件是：`main.py`、`detector.py`、`door_strategy.py`、`minimap_astar.py`、`minimap_nav.py`、`game.py`。
`olaplug` 目录虽然很大，但不是这条主刷图链里真正执行的核心代码，所以这次仍然放在附录说明，不抢主线。

目录

- 1. `main.py`：总入口
- 2. `detector.py`：YOLO 大画面检测
- 3. `door_strategy.py`：门选择评分
- 4. `minimap_astar.py`：A 星寻路
- 5. `minimap_nav.py`：小地图识别与路线生成
- 6. `game.py`：动作控制、打怪、拾取、过门、卡住恢复
- 7. `minimap_route_demo.py`：导航测试脚本

第一章：`main.py`

这个文件负责什么：它是整个刷图程序的总入口。负责找窗口、摆窗口、截图、调用 YOLO 检测、调用小地图导航，再把结果交给 `Game` 去执行。

片段 1：文件头、导入、路径兼容和窗口常量

`main.py` 顶部

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
@File      : main.py
@Desc      : Win32-based DNF capture loop
"""

from __future__ import annotations

import pathlib
import time
from typing import Optional, Tuple

import cv2
import numpy as np
import pyautogui
import win32con
import win32gui
from loguru import logger

from dnf.game import Game
from dnf.minimap_nav import MiniMapNavigator

temp = pathlib.PosixPath
pathlib.PosixPath = pathlib.WindowsPath

from dnf.detector import Detector

WINDOW_OFFSET_X = 10
WINDOW_OFFSET_Y = 10
WINDOW_TITLE_KEYWORD = "地下城与勇士"
WINDOW_CLASS_NAMES = {"地下城与勇士", "地下城与勇士创新世纪"}
```

这一段的作用

这一段先把主程序需要的库全部导进来。 `cv2` 用来处理图像， `numpy` 接收截图数组， `pyautogui` 负责截图， `win32gui` 与 `win32con` 负责 Windows 窗口操作， `Game` 是动作层， `MiniMapNavigator` 是导航层， `Detector` 是 YOLO 检测层。

执行逻辑

这里还有一个路径兼容小技巧：把 `pathlib.PosixPath` 临时替换成 `WindowsPath`，是为了避免某些依赖在 Windows 下读路径时出问题。下面的几个

常量则是后面找 DNF 窗口和摆窗口位置时要用的过滤条件和预设位置。

它是干什么的

这属于程序的基础设施区。还没开始刷图，但后面整个链条都会依赖这一段准备好的模块和常量。

片段 2：寻找 DNF 窗口

函数 `_find_dnf_window()`

```
def _find_dnf_window() -> int:
    candidates = []

    def _enum_windows(hwnd: int, _: object) -> None:
        if not win32gui.IsWindowVisible(hwnd):
            return

        title = win32gui.GetWindowText(hwnd).strip()
        class_name = win32gui.GetClassName(hwnd).strip()
        if not title and not class_name:
            return

        title_match = WINDOW_TITLE_KEYWORD in title
        class_match = class_name in WINDOW_CLASS_NAMES or WINDOW_TITLE_KEYWORD
        in_class_name = class_name in WINDOW_CLASS_NAMES
        if not title_match and not class_match:
            return

        try:
            left, top, right, bottom = win32gui.GetWindowRect(hwnd)
            area = max(0, right - left) * max(0, bottom - top)
        except win32gui.error:
            area = 0

        candidates.append((area, hwnd, title, class_name))

    win32gui.EnumWindows(_enum_windows, None)
    if not candidates:
        raise RuntimeError("没有找到 DNF 窗口")

    candidates.sort(key=lambda item: item[0], reverse=True)
    hwnd = candidates[0][1]
```

```
logger.info(  
    "绑定窗口: hwnd={}, title={}, class={}",  
    hwnd,  
    candidates[0][2],  
    candidates[0][3],  
)  
return hwnd
```

这一段的作用

这段代码负责在系统所有可见窗口中，把 DNF 窗口找出来，并返回它的句柄 `hwnd`。这个句柄可以理解为窗口在系统里的身份证。

执行逻辑

程序先遍历全部窗口，再按顺序做过滤：不可见的不要，标题和类名都为空的不要，标题和类名都不匹配 DNF 关键词的也不要。只有像 DNF 的窗口才会继续读取大小，并按面积加入候选列表。最后从候选里挑面积最大的那个窗口作为真正目标。

它是干什么的

它本质上不是后台绑定，而是确定“我要盯住哪一个窗口”。这一点很重要，因为后面的截图都要以这个窗口为中心。

片段 3：把窗口恢复并摆到固定位置

函数 `_place_window()`

```
def _place_window(hwnd: int) -> None:  
    win32gui.ShowWindow(hwnd, win32con.SW_RESTORE)  
    left, top, right, bottom = win32gui.GetWindowRect(hwnd)  
    width = max(1, right - left)  
    height = max(1, bottom - top)  
    win32gui.SetWindowPos(  
        hwnd,  
        win32con.HWND_TOPMOST,  
        WINDOW_OFFSET_X,  
        WINDOW_OFFSET_Y,  
        width,  
        height,  
        win32con.SWP_SHOWWINDOW,  
    )
```

这一段的作用

找到窗口后，程序会把它恢复出来并移动到左上角附近的固定位置，这样截图更稳定。

执行逻辑

`SW_RESTORE` 是把窗口从最小化状态恢复。然后读取窗口当前大小，再调用 `SetWindowPos()` 把它放到预设偏移位置，并设置成置顶显示。

它是干什么的

自动化最怕窗口位置乱变，这一步就是为了让后面的截图区域稳定下来。

片段 4：计算客户区并截图

函数 `_get_client_region()` 与 `_capture_client()`

```
def _get_client_region(hwnd: int) -> Tuple[int, int, int, int]:
    left_top = win32gui.ClientToScreen(hwnd, (0, 0))
    client_rect = win32gui.GetClientRect(hwnd)
    left = int(left_top[0])
    top = int(left_top[1])
    width = int(client_rect[2] - client_rect[0])
    height = int(client_rect[3] - client_rect[1])
    if width <= 0 or height <= 0:
        raise RuntimeError("获取游戏客户区失败")
    return left, top, width, height

def _capture_client(hwnd: int) -> Tuple[np.ndarray, Tuple[int, int]]:
    left, top, width, height = _get_client_region(hwnd)
    img = pyautogui.screenshot(region=(left, top, width, height))
    img_np = np.array(img)
    return img_np, (width, height)
```

这一段的作用

这段代码真正把游戏画面截下来。注意这里截的是客户区，也就是真正显示游戏内容的区域，不包括标题栏和边框。

执行逻辑

程序先用 `ClientToScreen()` 得到客户区左上角的屏幕坐标，再用 `GetClientRect()` 得到客户区宽高。之后用这组矩形参数调用 `pyautogui.screenshot()` 截图，并转成 `numpy` 数组，供 OpenCV 和 YOLO 使用。

它是干什么的

这一步就是给整个识别链路喂原始视觉输入。没有截图，后面的识别和导航都无从谈起。

片段 5：主函数初始化

函数 `main()` 开头

```
def main() -> None:
    device_type = ""
    detector = Detector(device_type)
    navigator = MiniMapNavigator("generic")

    hwnd = _find_dnf_window()
    _place_window(hwnd)
```

这一段的作用

主函数开始时，会先把大画面检测器和小地图导航器都准备好，然后找并摆好 DNF 窗口。

执行逻辑

`device_type = ""` 代表让检测器自己决定用 GPU 还是 CPU。

`MiniMapNavigator("generic")` 表示当前默认按 `generic` 地图规格处理小地图。

它是干什么的

这是进入主循环前的一次性准备步骤。模型和导航器只需要初始化一次，不必每帧都重建。

片段 6：主循环里截图、检测和导航

函数 `main()` 中段

```
while True:
    start_time = time.time()
    try:
        img_np, (width, height) = _capture_client(hwnd)
    except Exception as exc:
        logger.exception("截图失败: {}", exc)
        time.sleep(0.2)
        continue

    img, obj = detector.detect(img_np)
    logger.info("obj: {}", obj)

    frame_bgr = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR)
    route_snapshot = navigator.build_route_snapshot(frame_bgr, obj or [])
    route_direction = (
        route_snapshot.next_room_direction.upper()
        if route_snapshot.next_room_direction
        else None
    )
```

这一段的作用

这是主循环的核心前半段：先截图，再做大画面检测，再做小地图导航，最终拿到房间级方向提示。

执行逻辑

先截图；若截图失败，则记录异常、稍作等待后直接重来。截图成功后，把图送进 YOLO 检测器，得到检测框和可视化图。然后把原图转成 BGR，送进导航器。导航器会把小地图识别结果和大画面门、玩家信息结合起来，得到当前房间、目标房间和下一步方向。

它是干什么的

这一段相当于“看图并思考路线”，它只负责得出判断，不负责实际按键。

片段 7：把方向交给动作层执行函数 `main()` 结尾

```
        logger.info(
            "route: current={}, boss={}, query={}, elite={}, down={},
direction={}, door={}, scores={}",
            route_snapshot.current_room,
            route_snapshot.boss_room,
            route_snapshot.query_room,
            route_snapshot.elite_room,
            route_snapshot.down_room,
            route_direction,
            route_snapshot.selected_door_center,
            route_snapshot.debug_scores,
        )

    game = Game(obj, width, height, route_direction)
    game.run()

    display = cv2.resize(img, (640, 360))
    cv2.imshow("112233", display)
    logger.info("处理时间: {}", time.time() - start_time)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

这一段的作用

这里把识别和导航的结果交给动作层 `Game`，让动作层决定这一帧该打怪、捡东西还是朝门移动。

执行逻辑

先输出路线调试信息，再创建 `Game` 对象，并把检测结果、画面尺寸和路线方向交给它。`game.run()` 会真正执行本帧行为。后面还会显示一个缩放后的检测窗口，方便开发者肉眼检查识别效果。

它是干什么的

`main.py` 在这里完成了自己的使命：把“看”和“想”的结果，转交给“动”的部分去执行。

第二章： detector.py

这个文件负责什么： 用 YOLO 模型识别大画面中的玩家、怪物、Boss、门、金币和掉落物，是程序的第一只眼睛。

片段 1：初始化检测器

类 `Detector` 的 `__init__`

```
class Detector:
    def __init__(self, device_type: str = ""):
        self.img_size = 640
        self.conf_thres = 0.75
        self.iou_thres = 0.45
        self.hide_labels = False
        self.hide_conf = False

        self.device = device_type or ("0" if torch.cuda.is_available() else
"cpu")

        self.weights = str(Path(__file__).resolve().parent / "shzn.pt")
        self.model = YOLO(self.weights)
        self.names = self.model.names
        self.colors = [[random.randint(0, 255) for _ in range(3)] for _ in
range(len(self.names))]
```

这一段的作用

这段代码负责把 YOLO 模型真正加载起来，并设置推理参数。

执行逻辑

`img_size` 指输入模型的尺寸，`conf_thres` 是置信度阈值，`iou_thres` 是 NMS 去重阈值。程序会自动优先使用 GPU，不行再退回 CPU。模型文件是本地的 `shzn.pt`。

它是干什么的

这是检测器的开机步骤。没有这一步，程序后面就不知道画面里哪些框是怪、哪些框是门。

片段 2：标准化标签和坐标格式

两个静态工具函数

```
@staticmethod
def _normalize_label(label: str) -> str:
    label = str(label).strip().lower()
    if "door" in label:
        return "door"
    return label

@staticmethod
def _xyxy_to_xywh_top_left(xyxy: np.ndarray) -> List[float]:
    x1, y1, x2, y2 = [float(v) for v in xyxy]
    return [x1, y1, max(0.0, x2 - x1), max(0.0, y2 - y1)]
```

这一段的作用

把模型吐出来的数据整理成统一格式。标签统一成小写字母，门相关的标签统一叫 `door`；同时把坐标框从 `xyxy` 转成 `xywh`。

执行逻辑

这样做后，后面的动作层和导航层就不需要各自重复解析坐标和标签了。

它是干什么的

这是一个典型的数据清洗层，目的是让后面的模块使用同样的输入结构。

片段 3：检测函数入口

函数 `detect()` 开头

```
def detect(self, img0: np.ndarray) -> Tuple[np.ndarray | None, list | None]:
    if img0 is None:
        return None, None

    img_bgr = cv2.cvtColor(img0, cv2.COLOR_RGB2BGR)
    result = self.model.predict(
        source=img_bgr,
        imgsz=self.img_size,
        conf=self.conf_thres,
        iou=self.iou_thres,
```

```
device=self.device,  
verbose=False,  
)[0]
```

这一段的作用

这段代码真正开始对当前截图做 YOLO 目标检测。

执行逻辑

如果输入图片为空，就直接返回空结果，防止程序出错。否则先把 RGB 转成 BGR，再把图像送进模型预测。

它是干什么的

从这里开始，程序就真正“看见”画面了。

片段 4：整理检测结果

函数 `detect()` 后半段

```
annotator = Annotator(img_bgr.copy(), line_width=3, example=self.names)  
obj = []  
  
if result.bboxes is not None and len(result.bboxes) > 0:  
    boxes_xyxy = result.bboxes.xyxy.cpu().numpy()  
    boxes_conf = result.bboxes.conf.cpu().numpy()  
    boxes_cls = result.bboxes.cls.cpu().numpy().astype(int)  
  
    for xyxy, conf, cls in zip(boxes_xyxy, boxes_conf, boxes_cls):  
        raw_label = self.names[int(cls)]  
        label = self._normalize_label(raw_label)  
        xywh = self._xyxy_to_xywh_top_left(xyxy)  
        conf = round(float(conf), 2)  
  
        annotator.box_label(  
            xyxy,  
            raw_label if self.hide_conf else f"{raw_label} {conf:.2f}",  
            color=tuple(self.colors[int(cls)]),  
        )  
        obj.append({label: {"xywh": xywh, "conf": conf}})  
  
return annotator.result(), obj
```

这一段的作用

把模型原始输出整理成程序更容易使用的对象列表，同时顺手把检测框画到图上。

执行逻辑

每个框都会被拆成类别、置信度和位置。类别经过标准化，位置经过坐标转换，最后存入 `obj` 列表。这个 `obj` 就是后面打怪、捡东西、过门的直接依据。

它是干什么的

前者是给程序看的结构化结果，后者是给人看的调试可视化图。

第三章： `door_strategy.py`

这个文件负责什么： 当 A 星告诉程序“下一间房在右边”时，这里负责在当前画面多扇门里找出最像“右门”的那一扇。

片段 1：门数据结构和方向取反

文件顶部

```
@dataclass
class DoorCandidate:
    bbox: Sequence[float]
    center: Point

def opposite_direction(direction: Optional[str]) -> Optional[str]:
    mapping = {
        "left": "right",
        "right": "left",
        "up": "down",
        "down": "up",
    }
    return mapping.get(direction)
```

这一段的作用

先把门封装成统一结构，再提供一个“求反方向”的工具函数。

执行逻辑

后面选门时既要知道门中心点，也要知道上一次路线方向是不是反向掉头，所以这两个工具都很基础。

它是干什么的

这是门策略的底层数据准备层。

片段 2：计算偏移和方向匹配

两个小函数

```
def _axis_offsets(player: Point, door: Point) -> Tuple[float, float]:
    dx = door[0] - player[0]
    dy = door[1] - player[1]
    return dx, dy

def _direction_match(expected_direction: str, dx: float, dy: float, margin:
float) -> bool:
    if expected_direction == "right":
        return dx > margin
    if expected_direction == "left":
        return dx < -margin
    if expected_direction == "up":
        return dy < -margin
    if expected_direction == "down":
        return dy > margin
    return True
```

这一段的作用

判断一扇门相对玩家到底偏在哪边，以及它算不算符合当前期望方向。

执行逻辑

`margin` 是容差。比如期望向右时，门必须明显在玩家右边，不能只是刚好挨着中线。

它是干什么的

这是把 A 星方向和屏幕中门位置联系起来的的第一步。

片段 3: 核心选门算法

函数 `choose_best_door()`

```
def choose_best_door(
    doors: Sequence[DoorCandidate],
    player_center: Point,
    expected_direction: Optional[str],
    last_direction: Optional[str] = None,
    margin: float = 16.0,
) -> Optional[DoorCandidate]:
    if not doors:
        return None
    if expected_direction is None:
        return min(doors, key=lambda item: abs(item.center[0] -
player_center[0]) + abs(item.center[1] - player_center[1]))

    reverse_direction = opposite_direction(last_direction)
    candidates = []
    fallback = []

    for door in doors:
        dx, dy = _axis_offsets(player_center, door.center)
        abs_dx = abs(dx)
        abs_dy = abs(dy)
        match = _direction_match(expected_direction, dx, dy, margin)

        if expected_direction in ("left", "right"):
            forward = dx if expected_direction == "right" else -dx
            sideways = abs_dy
        else:
            forward = dy if expected_direction == "down" else -dy
            sideways = abs_dx

        reverse_penalty = 0.0
        if reverse_direction and expected_direction == reverse_direction:
            reverse_penalty = 2000.0

        score = (-forward + reverse_penalty, sideways, abs_dx + abs_dy)
        if match:
            candidates.append((score, door))
        fallback.append((score, door))

    if candidates:
```

```
candidates.sort(key=lambda item: item[0])
return candidates[0][1]

fallback.sort(key=lambda item: item[0])
return fallback[0][1]
```

这一段的作用

这是门选择的核心算法。它会对每扇门打分，再选出最适合当前路线方向的那一扇。

执行逻辑

没有门时直接返回空；没有方向提示时，选最近门。有方向提示时，对每扇门计算三个维度：前进量、侧向偏移、总距离，同时还会给明显的反方向门加一个很大的惩罚。匹配方向的门会进入优先候选；全部门都会进入兜底候选。最终优先从匹配候选中取分数最好的，没有才用兜底。

它是干什么的

它的本质不是复杂数学，而是一个很实用的启发式评分器：尽量选既符合方向、又不偏、又不远的门。

第四章：minimap_astar.py

这个文件负责什么： 做房间级 A 星寻路。注意它输出的是“下一间房往哪边走”，不是房间内部的像素级路径。

片段 1：节点结构与曼哈顿启发

文件顶部

```
@dataclass(order=True)
class _Node:
    f: int
    position: GridPoint
    g: int = 0
    h: int = 0
    parent: Optional["_Node"] = None
```

```
def heuristic(a: GridPoint, b: GridPoint) -> int:
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

这一段的作用

定义 A 星节点和启发函数。 $f = g + h$ 是 A 星最核心的评分逻辑。

执行逻辑

g 表示已经走过的代价， h 是到终点的估计代价，这里用曼哈顿距离，因为地图只考虑上下左右移动。

它是干什么的

这是整个寻路算法的地基。

片段 2：邻居扩展顺序

函数 `_neighbors()`

```
def _neighbors(position: GridPoint, grid: Sequence[Sequence[int]], priority:
str):
    x, y = position
    directions = {
        "right": ((0, 1), (1, 0), (-1, 0), (0, -1)),
        "left": ((0, -1), (-1, 0), (1, 0), (0, 1)),
        "up": ((-1, 0), (0, -1), (0, 1), (1, 0)),
        "down": ((1, 0), (0, 1), (0, -1), (-1, 0)),
        "default": ((0, 1), (1, 0), (-1, 0), (0, -1)),
    }
```

这一段的作用

决定当前节点应该优先往哪个方向扩展邻居。比如优先级是右，就先看右再看别的方向。

执行逻辑

这不会改变最短路的正确性，但在多条路径一样短时，会影响程序更偏向哪一条。

它是干什么的

你可以把它理解成“同分情况下的路线偏好”。

片段 3: A 星主循环

函数 `a_star()`

```
def a_star(grid, start, end, priority="right"):
    if start == end:
        return [start]

    open_list = []
    start_node = _Node(f=0, position=start, g=0, h=heuristic(start, end),
parent=None)
    start_node.f = start_node.g + start_node.h
    heapq.heappush(open_list, start_node)
    closed = set()
    best_g = {start: 0}

    while open_list:
        current = heapq.heappop(open_list)
        if current.position in closed:
            continue
        closed.add(current.position)

        if current.position == end:
            path = []
            node = current
            while node is not None:
                path.append(node.position)
                node = node.parent
            return list(reversed(path))

        for next_pos in _neighbors(current.position, grid, priority):
            next_g = current.g + 1
            if next_g >= best_g.get(next_pos, 10 ** 9):
                continue
            best_g[next_pos] = next_g
            next_h = heuristic(next_pos, end)
            heapq.heappush(
                open_list,
                _Node(
                    f=next_g + next_h,
                    position=next_pos,
                    g=next_g,
                    h=next_h,
                    parent=current,
```

```
        ),
    )
    return None
```

这一段的作用

这就是完整的 A 星搜索过程：不断取出当前最优节点，直到找到终点并回溯出整条路径。

执行逻辑

开放列表里永远是“可能值得继续探索”的节点。每次取出 `f` 最小的那个，扩展邻居；如果到了终点，就沿着 `parent` 链一路回溯生成路径。

它是干什么的

它在整套程序里负责房间级最短路径搜索，是导航层最核心的算法。

片段 4：只取路径第一步方向

函数 `judge_direction()` 与 `next_direction()`

```
def judge_direction(starting_point: GridPoint, end_point: GridPoint) -> Optional[str]:
    x1, y1 = starting_point
    x2, y2 = end_point
    if x2 > x1:
        return "down"
    if x2 < x1:
        return "up"
    if y2 > y1:
        return "right"
    if y2 < y1:
        return "left"
    return None

def next_direction(grid, start, end, priority="right") -> Optional[str]:
    path = a_star(grid, start, end, priority=priority)
    if not path or len(path) < 2:
        return None
    return judge_direction(path[0], path[1])
```

这一段的作用

把整条路径压缩成一个最重要的信息：下一步往哪边走。

执行逻辑

当前系统并不把整条房间路径都交给动作层，而是只取前两点之间的方向，这样动作层只需要关心“下一间房在上、下、左、右哪边”。

它是干什么的

这是 A 星和动作层衔接的出口。A 星不直接控制角色移动，只负责给出方向提示。

第五章： minimap_nav.py

这个文件负责什么： 它是小地图导航中心：裁小地图、匹配图标、识别当前房间和目标房间、调用 A 星，并尝试把房间方向关联到当前画面中的门。

片段 1：地图规格与路线快照结构

顶部数据结构

```
@dataclass
class MapSpec:
    name: str
    crop_rect_1067: Rect
    minimap_width: int
    minimap_height: int
    rows: int
    cols: int
    room_grid: List[List[int]]

@dataclass
class RouteSnapshot:
    current_room: Optional[Point]
    boss_room: Optional[Point]
    query_room: Optional[Point]
    elite_room: Optional[Point]
    down_room: Optional[Point]
    next_room_direction: Optional[str]
```

```
selected_door_center: Optional[Tuple[float, float]]
debug_scores: Optional[Dict[str, float]] = None
```

这一段的作用

`MapSpec` 描述地图该怎么被理解, `RouteSnapshot` 描述导航器本轮最终算出来的完整结果。

执行逻辑

前者更像配置表, 后者更像每一帧的输出结果包。

它是干什么的

这是导航层的数据骨架。

片段 2: 地图字典

MAP_SPECS

```
def _all_walkable(rows: int, cols: int) -> List[List[int]]:
    return [[0 for _ in range(cols)] for _ in range(rows)]
```

```
MAP_SPECS = {
    "generic": MapSpec(
        name="generic",
        crop_rect_1067=(893, 52, 1055, 142),
        minimap_width=162,
        minimap_height=90,
        rows=5,
        cols=9,
        room_grid=_all_walkable(5, 9),
    ),
    "haibolun": MapSpec(
        name="haibolun",
        crop_rect_1067=(966, 52, 1056, 124),
        minimap_width=90,
        minimap_height=72,
        rows=4,
        cols=5,
        room_grid=_all_walkable(4, 5),
    ),
}
```

```
),  
}
```

这一段的作用

定义不同地图模板的裁剪区域和房间网格规模。

执行逻辑

当前这份代码中的 `room_grid` 基本是全 0，也就是默认所有房间位置都可通行。

它是干什么的

它告诉导航器“这张地图应该怎么切、怎么分格子、A 星要在哪张网格上跑”。

片段 3：初始化导航器并加载模板

类 `MiniMapNavigator` 的 `__init__`

```
def __init__(self, map_name: str = "generic", assets_dir: Optional[Path] =  
None, threshold: float = 0.7):  
    if map_name not in MAP_SPECS:  
        map_name = "generic"  
    self.map_name = map_name  
    self.spec = MAP_SPECS[map_name]  
    self.threshold = threshold  
    self.assets_dir = assets_dir or (Path(__file__).resolve().parent / "res")  
    self.templates = {  
        "hero": self._load_template("map_hero.png"),  
        "boss": self._load_template("map_boss.png"),  
        "query": self._load_template("map_query.png"),  
        "query1": self._load_template("map_query1.png"),  
        "query3": self._load_template("map_query3.png"),  
        "query4": self._load_template("map_query4.png"),  
        "query5": self._load_template("map_query5.png"),  
        "elite": self._load_template("map_elite.png"),  
        "special": self._load_template("map_special_room.png"),  
    }  
    down_template = self._load_template_optional("map_down.png")  
    if down_template is not None:  
        self.templates["down"] = down_template  
    self.last_direction: Optional[str] = None
```

这一段的作用

把小地图导航器准备好，并把所有图标模板一次性读入内存。

执行逻辑

先确定当前地图规格，再加载英雄图标、Boss 图标、问号房图标、精英房图标等。

`last_direction` 用来记住上一次路线方向，后面选门时会避免掉头。

它是干什么的

它相当于给导航器装上“识别字典”。

片段 4：换算小地图裁剪区域

函数 `_scaled_crop_rect()` 与 `extract_minimap()`

```
def _scaled_crop_rect(self, frame: np.ndarray) -> Rect:
    frame_h, frame_w = frame.shape[:2]
    x1, y1, x2, y2 = self.spec.crop_rect_1067
    scale_x = frame_w / 1067.0
    scale_y = frame_h / 600.0
    return (
        int(round(x1 * scale_x)),
        int(round(y1 * scale_y)),
        int(round(x2 * scale_x)),
        int(round(y2 * scale_y)),
    )

def extract_minimap(self, frame: np.ndarray) -> np.ndarray:
    x1, y1, x2, y2 = self._scaled_crop_rect(frame)
    return frame[y1:y2, x1:x2]
```

这一段的作用

把整张截图中的小地图裁出来，同时对不同分辨率做比例适配。

执行逻辑

地图配置里的裁剪矩形是按基准分辨率写死的，这里会按当前截图尺寸进行缩放换算，保证不同窗口大小下都能裁到正确的小地图区域。

它是干什么的

这是小地图识别的第一步，没有正确裁图，模板匹配全部都会偏掉。

片段 5：模板匹配核心

函数 `_match_template()` 与 `_match_template_debug()`

```
def _match_template(self, image: np.ndarray, template: np.ndarray, threshold:
Optional[float] = None):
    threshold = self.threshold if threshold is None else threshold
    image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
    result = cv2.matchTemplate(image_gray, template_gray,
cv2.TM_CCOEFF_NORMED)
    ys, xs = np.where(result >= threshold)
    matches = []
    h, w = template_gray.shape[:2]
    for y, x in zip(ys, xs):
        matches.append((x + w / 2.0, y + h / 2.0))
    return matches
```

这一段的作用

这是小地图识别的核心：拿一张模板图，在小地图上滑动比较，看哪里最像它。

执行逻辑

先转灰度，再用 `cv2.matchTemplate()` 做相关性匹配，最后把所有超过阈值的位置换算成模板中心点。

它是干什么的

这是导航器的“视觉识别引擎”，专门认小地图上的房间图标。

片段 6：像素位置换成房间坐标

函数 `compute_room_id()`

```
def compute_room_id(self, x: float, y: float, minimap_image: np.ndarray) ->
Point:
    height, width = minimap_image.shape[:2]
    cell_w = width / self.spec.cols
    cell_h = height / self.spec.rows
    row = int(y // cell_h)
    col = int(x // cell_w)
    row = max(0, min(self.spec.rows - 1, row))
```

```
col = max(0, min(self.spec.cols - 1, col))
return row, col
```

这一段的作用

把图标中心的像素坐标转换成房间网格坐标。

执行逻辑

根据小地图总宽高和行列数，先算每个房间格子的尺寸，再用整除运算得出图标落在哪一行哪一列。

它是干什么的

A 星不吃像素坐标，只吃房间坐标，所以必须做这一步转换。

片段 7：识别当前房间、Boss 房、问号房等

函数 `detect_room_markers()`

```
def detect_room_markers(self, frame: np.ndarray) -> Dict[str,
Optional[Point]]:
    minimap = self.extract_minimap(frame)
    current_room = None
    boss_room = None
    query_room = None
    elite_room = None
    down_room = None

    hero_matches = self._match_template(minimap, self.templates["hero"])
    if hero_matches:
        current_room = self.compute_room_id(*hero_matches[0], minimap)

    boss_matches = self._match_template(minimap, self.templates["boss"])
    if boss_matches:
        boss_room = self.compute_room_id(*boss_matches[0], minimap)

    query_matches = self._match_template(minimap, self.templates["query"])
    if not query_matches:
        query_matches = self._match_template(minimap,
self.templates["query1"])
    if not query_matches:
        query_matches = self._match_template(minimap,
self.templates["query3"])
```



```
if not query_matches:
    query_matches = self._match_template(minimap,
self.templates["query4"])
if not query_matches:
    query_matches = self._match_template(minimap,
self.templates["query5"])
if query_matches:
    query_room = self.compute_room_id(*query_matches[0], minimap)
```

这一段的作用

把小地图上最重要的房间标记都识别出来。

执行逻辑

英雄图标对应当前房间，Boss 图标对应 Boss 房。问号房比较特殊，代码准备了多个不同样式模板，匹配到任意一个就算找到。

它是干什么的

它在告诉程序：我现在在哪，值得去的房间又在哪。

片段 8：选择目标房间

函数 `_pick_target_room()`

```
def _pick_target_room(
    self,
    current_room,
    query_room,
    down_room,
    elite_room,
    boss_room,
    prefer_special_room: bool,
) -> Optional[Point]:
    candidates = []
    if prefer_special_room:
        candidates.extend([query_room, down_room, elite_room, boss_room])
    else:
        candidates.append(boss_room)

    for room in candidates:
        if room is None:
            continue
```

```
        if current_room is not None and room == current_room:
            continue
        return room
    return None
```

这一段的作用

从多个候选房间中选出当前最值得前往的目标房。

执行逻辑

当前默认会优先走问号房、下层房、精英房，最后才是 Boss 房。找到第一个有效且不是当前房间的目标，就直接作为终点。

它是干什么的

这一步是在决定“A 星应该从哪算到哪”。

片段 9：生成完整路线快照

函数 `build_route_snapshot()`

```
def build_route_snapshot(self, frame_bgr, detection_objects,
    prefer_special_room=True) -> RouteSnapshot:
    room_info = self.detect_room_markers(frame_bgr)
    current_room = room_info["current_room"]
    boss_room = room_info["boss_room"]
    query_room = room_info["query_room"]
    elite_room = room_info["elite_room"]
    down_room = room_info["down_room"]

    target_room = self._pick_target_room(
        current_room=current_room,
        query_room=query_room,
        down_room=down_room,
        elite_room=elite_room,
        boss_room=boss_room,
        prefer_special_room=prefer_special_room,
    )

    next_room_direction = None
    if current_room is not None and target_room is not None:
```

```
next_room_direction = next_direction(deepcopy(self.spec.room_grid),
current_room, target_room, priority="right")
```

这一段的作用

这段代码把小地图识别结果和 A 星真正接起来，得出“下一步往哪间房走”。

执行逻辑

先识别房间，再选目标房，再把当前房间和目标房丢给 `next_direction()`，从而得到上、下、左、右之一。

它是干什么的

这是小地图视觉信息变成路线方向的关键结合点。

片段 10：把方向继续关联到门

函数 `build_route_snapshot()` 后半段

```
selected_door_center = None
if next_room_direction:
    player_center = None
    for item in detection_objects:
        if "player" in item:
            x, y, w, h = item["player"]["xywh"]
            player_center = (x + w / 2.0, y + h / 2.0)
            break
    if player_center is not None:
        selected_door = choose_best_door(
            self._door_candidates_from_objects(detection_objects),
            player_center=player_center,
            expected_direction=next_room_direction,
            last_direction=self.last_direction,
        )
    if selected_door is not None:
        selected_door_center = selected_door.center
        self.last_direction = next_room_direction
```

这一段的作用

在已经知道房间方向以后，进一步尝试在当前画面里选出一扇具体门。

执行逻辑

程序先找玩家中心，再把所有门整理成门候选列表，然后调用前面讲过的门评分算法，选出最像目标方向的那扇门。

它是干什么的

这说明 A 星不是孤立存在的。它会继续影响当前房间的门选择。

第六章： game.py

这个文件负责什么： 这是刷图动作控制中心。决定这一帧究竟该打怪、拾取、过门，还是执行卡住恢复。

片段 1：类级状态与初始化阈值

类 Game 开头

```
class Game:
    _action_cache: Optional[str] = None
    _player: Optional[BoxDict] = None
    _pre_player: Optional[BoxDict] = None
    _index = 0
    _player_missing_count = 0
    _position_history: List[Tuple[float, float]] = []
    _stuck_count = 0
    _last_recover_time = 0.0
    _recover_step = 0
    _last_player_center: Optional[Tuple[float, float]] = None
    _room_entry_position: Optional[Tuple[float, float]] = None
    _room_entry_time = 0.0
    _flow_direction: Optional[str] = None
    _missing_player_recover_threshold = 10
    _route_search_direction: Optional[str] = None
    _route_search_phase = 0
    _route_search_start: Optional[Tuple[float, float]] = None

    def __init__(self, obj: Sequence[dict], width: int, height: int,
direction):
        self._obj = obj or []
        self._width = width
        self._height = height
```

```
self._player_xywh: Optional[List[float]] = None
self._attack_x = 100
self._attack_y = 100
self._move_x = 20
self._move_y = 20
self._direction = direction
```

这一段的作用

这里定义了动作系统跨帧记忆的所有状态，以及攻击范围、拾取范围这些重要阈值。

执行逻辑

像上一帧动作、最近几帧位置、卡住计数、入房时间等都保存在类变量里，因为它们需要在多帧之间连续存在。

它是干什么的

这是动作系统的“记忆仓库”。自动化不是一帧就结束，所以一定要有跨帧状态。

片段 2：读取方向提示和检测对象

三个基础工具函数

```
def _current_door_direction(self) -> Optional[str]:
    if isinstance(self._direction, (list, tuple)):
        if not self._direction:
            return None
        if Game._index >= len(self._direction):
            return str(self._direction[-1]).upper()
        return str(self._direction[Game._index]).upper()
    if isinstance(self._direction, str) and self._direction.strip():
        return self._direction.strip().upper()
    return None

def _get_cls(self, cls_name: str) -> Optional[BoxDict]:
    for item in self._obj:
        if cls_name in item:
            return item[cls_name]
    return None

def _get_clss(self, cls_name: str) -> List[BoxDict]:
    result = []
    for item in self._obj:
```

```
        if cls_name in item:
            result.append(item[cls_name])
    return result
```

这一段的作用

统一读取当前路线方向，以及从检测结果里取出某一类目标。

执行逻辑

`_get_cls()` 取一个，`_get_cls()` 取一类全部对象。方向可以是单字符串，也可以是多步方向列表。

它是干什么的

它们是后续所有动作判断最常用的读数接口。

片段 3：释放按键和距离函数

基础动作工具

```
def _release_direction_keys(self) -> None:
    for key in ("up", "down", "left", "right"):
        pydirectinput.keyUp(key)

def _release_cached_action(self) -> None:
    if not Game._action_cache:
        self._release_direction_keys()
        return
    for action in Game._action_cache.strip().split("_"):
        pydirectinput.keyUp(action.lower())
    self._release_direction_keys()
    Game._action_cache = None

def _distance(self, a, b) -> float:
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5
```

这一段的作用

一组很基础但很关键的工具：松开方向键、松开缓存动作、计算两点距离。

执行逻辑

自动化最怕按键黏住，所以每次改变动作前都要有可靠的释放逻辑。

它是干什么的

这是动作控制层的安全基础。

片段 4：推断入房方向与更新入口点

房间入口保护逻辑

```
def _infer_flow_direction_from_spawn(self, player_center):
    x, y = player_center
    edge_scores = {
        "LEFT": x,
        "RIGHT": self._width - x,
        "UP": y,
        "DOWN": self._height - y,
    }
    nearest_edge = min(edge_scores, key=edge_scores.get)
    opposite = {
        "LEFT": "RIGHT",
        "RIGHT": "LEFT",
        "UP": "DOWN",
        "DOWN": "UP",
    }
    if edge_scores[nearest_edge] > min(self._width, self._height) * 0.28:
        return None
    return opposite[nearest_edge]
```

这一段的作用

根据角色刚进房时靠近哪个边缘，推断他是从哪个门进来的，并反推出推进方向。

执行逻辑

如果角色靠近左边缘，就认为是从左边进来，接下来通常应往右推进。阈值 `0.28` 是为了防止角色没靠边时胡乱推断。

它是干什么的

这是后面“防止刚进房就回头”这件事的依据。

片段 5：过滤回头门

入口保护函数

```
def _filter_backtrack_doors(self, doors, direction_hint):
    if not doors:
        return []
    if not Game._room_entry_position:
        return list(doors)
    now = time.time()
    protect_entry = now - Game._room_entry_time <= 3.0
    if not protect_entry:
        return list(doors)
    ...
    return filtered
```

这一段的作用

进房后的前几秒内，过滤掉太像“回头门”的门。

执行逻辑

如果门离入口太近，而且它又位于明显的回头方向，程序就会忽略它。这样可以显著减少来回穿门的抖动。

它是干什么的

这是刷图稳定性的关键保护逻辑之一。

片段 6：短按键、轻点方向和方向取反

动作底层函数

```
def _key_press(self, key: str) -> None:
    pydirectinput.keyDown(key)
    time.sleep(0.08)
    pydirectinput.keyUp(key)
    time.sleep(0.05)

def _tap_direction(self, direction: str, duration: float = 0.12) -> None:
    for action in direction.strip().split("_"):
        pydirectinput.keyDown(action.lower())
        time.sleep(duration)
    for action in direction.strip().split("_"):
```



```
pydirectinput.keyUp(action.lower())
time.sleep(0.04)
```

这一段的作用

封装基础输入动作：短按一个键，或者轻点一个方向。

执行逻辑

攻击和拾取都用短按，卡住恢复和试探移动则经常用轻点方向。

它是干什么的

这是动作层把高层意图变成真实键盘事件的底层执行器。

片段 7：选最近目标

函数 `_get_nearest()`

```
def _get_nearest(self, cls_objs, direct=None):
    nearest = None
    min_distance = float("inf")
    ...
    distance = (dx ** 2 + dy ** 2) ** 0.5
    if distance < min_distance:
        min_distance = distance
        nearest = {
            "xywh": [obj_x, obj_y, xywh[2], xywh[3]],
            "conf": item.get("conf", 0),
        }
    return nearest
```

这一段的作用

在一堆同类目标里，比如一堆怪、一堆门、一堆金币中，选离玩家最近的那个。

执行逻辑

程序会先把每个目标框转换成中心点，再按欧氏距离比较。如果传了方向限制，就只在某个方向上筛选。

它是干什么的

打怪、拾取和过门都大量依赖这个函数。

片段 8：把目标位置翻成方向字符串

函数 `_get_direction()`

```
def _get_direction(self, obj_box):
    if not self._player_xywh:
        return None
    dx = obj_box[0] - self._player_xywh[0]
    dy = obj_box[1] - self._player_xywh[1]
    ...
    return "RIGHT_UP" / "LEFT" / "DOWN" ...
```

这一段的作用

把“目标在玩家哪边”翻译成程序能执行的方向字符串，比如 `LEFT`、`RIGHT_DOWN`。

执行逻辑

它会比较横纵偏移的大小，并结合一个小容差阈值，决定目标是更像纯上下左右，还是更像斜方向。

它是干什么的

这是动作层的方向翻译器。后面的靠近怪、靠近门、靠近物品都要先经过这一步。

片段 9：真正的移动执行函数

函数 `_move()`

```
def _move(self, direction, is_slow=False, _action_cache=None, press_time=0.1,
release_time=0.1) -> str:
    if _action_cache and direction != _action_cache:
        for action in _action_cache.strip().split("_"):
            pydirectinput.keyUp(action.lower())

    for action in direction.strip().split("_"):
        pydirectinput.keyDown(action.lower())

    if not is_slow:
        time.sleep(press_time)
        for action in direction.strip().split("_"):
            pydirectinput.keyUp(action.lower())
```

```
time.sleep(release_time)
for action in direction.strip().split("_"):
    pydirectinput.keyDown(action.lower())

return direction
```

这一段的作用

把方向字符串真正变成按键动作。它是所有移动行为最后都会落到的函数。

执行逻辑

如果方向变了，就先松开旧方向，再按下新方向。普通移动带一点节奏控制，慢速移动则更平稳。

它是干什么的

这是动作层的总马达。

片段 10：判断是否卡住

函数 `_update_motion_state()`

```
def _update_motion_state(self, expect_motion: bool, player_visible: bool) -> bool:
    if not self._player_xywh:
        return False
    if not expect_motion or not player_visible:
        Game._position_history.clear()
        Game._stuck_count = 0
        return False
    ...
    if spread_x < 18 and spread_y < 18:
        Game._stuck_count += 1
    else:
        Game._stuck_count = 0
    return Game._stuck_count >= 2
```

这一段的作用

用最近几帧的位置历史来判断角色是不是卡住了。

执行逻辑

如果程序明明在移动，但最近几帧的横向和纵向扩散都很小，就增加卡住计数。连续两次满足这个条件，就认为真的卡住。

它是干什么的

这是一个典型的基于位置扩散范围的容差判定法。

片段 11：卡住恢复

函数 `_recover_from_stuck()`

```
def _recover_from_stuck(self, reason: str) -> None:
    now = time.time()
    if now - Game._last_recover_time < 1.2:
        return
    ...
    pattern = Game._recover_step % 4
    Game._recover_step += 1
    if pattern == 0:
        self._tap_direction(reverse_hint, 0.18)
        self._tap_direction("DOWN", 0.12)
    elif pattern == 1:
        self._tap_direction(reverse_hint, 0.18)
        self._tap_direction("UP", 0.12)
    elif pattern == 2:
        for step in ("RIGHT", "DOWN", "LEFT", "UP"):
            self._tap_direction(step, 0.1)
    else:
        self._tap_direction(reverse_hint, 0.22)
        self._tap_direction(current_hint if isinstance(current_hint, str) else
"RIGHT", 0.1)
```

这一段的作用

一旦判断卡住，程序就会启动一套脱困动作模式。

执行逻辑

它不是永远只做一件事，而是四种模式轮流试：反向后下、反向后上、小矩形绕步、反向再顺向。这样可以提高从卡点里脱出来的概率。

它是干什么的

这是动作层的自救系统。

片段 12：打怪

函数 `_kill_monster()`

```
def _kill_monster(self, obj_box) -> None:
    if not self._player_xywh:
        return

    if abs(obj_box[0] - self._player_xywh[0]) < self._attack_x and
abs(obj_box[1] - self._player_xywh[1]) < self._attack_y:
        direction = self._get_direction(obj_box)
        ...
        if face:
            self._key_press(face.lower())
            self._key_press("x")
            self._release_cached_action()
            return

    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction, is_slow=True,
        _action_cache=Game._action_cache)
```

这一段的作用

处理“去打这个怪”这件事。近了就打，远了就靠近。

执行逻辑

怪物进入攻击范围后，程序会先面向怪，再按 `x` 攻击。没进入攻击范围时，就慢慢朝怪方向走过去。

它是干什么的

当前这套战斗逻辑不复杂，就是非常直接的基础打怪模型。

片段 13：拾取与朝门移动

函数 `_pick_up()` 与 `_move_to_door()`

```
def _pick_up(self, obj_box) -> None:
    if not self._player_xywh:
        return
    if abs(obj_box[0] - self._player_xywh[0]) < self._move_x and
abs(obj_box[1] - self._player_xywh[1]) < self._move_y:
        self._key_press("f")
        self._release_cached_action()
        return
    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction, is_slow=True,
_action_cache=Game._action_cache)

def _move_to_door(self, obj_box) -> None:
    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction,
_action_cache=Game._action_cache)
```

这一段的作用

一个负责捡东西，一个负责朝门靠近。

执行逻辑

物品进入拾取范围就按 **f**，否则先靠近。门则不需要交互键，只要朝门移动即可。

它是干什么的

这两段说明动作层其实一直在重复同一套模式：判断是否足够近，近则交互，不近则移动接近。

片段 14：后备路线搜索与向上扫门

函数 `_fallback_up_search_move()` 与 `_fallback_route_move()`

```
def _fallback_up_search_move(self) -> None:
    ...
    if phase == 0:
        command = "DOWN"
    elif phase == 1:
        command = "RIGHT"
    elif phase == 2:
```

```
        command = "UP"
    else:
        command = "RIGHT"
    Game._action_cache = self._move(command, is_slow=True,
    _action_cache=Game._action_cache)

def _fallback_route_move(self, direction: str) -> None:
    normalized = direction.upper()
    if normalized == "DOWN":
        ...
    elif normalized == "UP":
        self._fallback_up_search_move()
        return
    ...
    Game._action_cache = self._move(normalized, is_slow=True,
    _action_cache=Game._action_cache)
```

这一段的作用

当当前帧没有明确目标，但路线提示还存在时，程序不会完全停住，而是执行一个受控的后备移动策略。

执行逻辑

向上方向最特殊，因为很多副本的上门不好对准，所以会先做一个分阶段扫门动作。向下则会根据角色当前横向位置微调成左下或右下。

它是干什么的

这是“看不清门时也尽量沿正确大方向探索”的启发式策略。

片段 15：主决策入口开头

函数 `run()` 前半段

```
def run(self) -> None:
    raw_player = self._get_cls("player")
    if raw_player is None:
        Game._player_missing_count += 1
        Game._player = Game._pre_player
    else:
        Game._player_missing_count = 0
        Game._player = raw_player
        Game._pre_player = raw_player
```

```
        if raw_player is None and Game._player_missing_count >=
Game._missing_player_recover_threshold:
            self._recover_missing_player()
            return

        if Game._player is None:
            return
```

这一段的作用

先确认这一帧能否定位到玩家。如果一时漏检，就临时沿用上一帧位置；如果连续很多帧都看不见玩家，就触发恢复。

执行逻辑

这样做可以减少 YOLO 偶发漏检造成的动作抖动。

它是干什么的

这是动作层进入决策前的定位校验步骤。

片段 16：更新玩家中心和卡住检测

函数 `run()` 中段

```
self._player_xywh = list(Game._player["xywh"])
self._player_xywh[0] = self._player_xywh[0] + self._player_xywh[2] / 2
self._player_xywh[1] = self._player_xywh[1] + self._player_xywh[3] / 2
logger.info("player center: {}", self._player_xywh[:2])
self._maybe_update_room_entry()

if self._update_motion_state(
    expect_motion=Game._action_cache is not None,
    player_visible=raw_player is not None,
):
    self._recover_from_stuck("movement_stalled")
    return
```

这一段的作用

把玩家框转换成中心点，更新入口状态，并立即检查是不是卡住。

执行逻辑

只要判断为卡住，就会立刻进入恢复流程，后面这帧不再继续战斗或过门判断。

它是干什么的

说明系统优先保证自己能动起来，再谈别的决策。

片段 17：战斗优先级

函数 `run()` 战斗部分

```
boss = self._get_class("boss")
if boss:
    nearest_boss = self._get_nearest(boss)
    if nearest_boss:
        self._kill_monster(nearest_boss["xywh"])
        return

monsters = self._get_class("monster")
if monsters:
    nearest_monster = self._get_nearest(monsters)
    if nearest_monster:
        self._kill_monster(nearest_monster["xywh"])
        return
```

这一段的作用

定义战斗优先级：先 Boss，后普通怪。

执行逻辑

一旦某一类怪物处理成功，当前帧立即结束，不会继续去捡东西或过门。

它是干什么的

这说明动作层是“单帧只做最重要的一件事”的设计。

片段 18：拾取优先级

函数 `run()` 拾取部分

```
goods = self._get_class("goods")
if goods:
    nearest_goods = self._get_nearest(goods)
```

```
if nearest_goods:
    self._pick_up(nearest_goods["xywh"])
    return

money = self._get_clss("money")
if money:
    nearest_money = self._get_nearest(money)
    if nearest_money:
        self._pick_up(nearest_money["xywh"])
    return
```

这一段的作用

没有怪可打时，程序会开始优先拾取掉落物和金币。

执行逻辑

和战斗一样，只要选中了一个最近物品并开始执行拾取，当前帧就结束。

它是干什么的

这体现出刷图行为顺序：清怪优先于捡东西，捡东西优先于过门。

片段 19：按路线方向选门并进门

函数 `run()` 过门部分

```
doors = self._get_clss("door")
if doors:
    direction_hint = self._current_door_direction()
    ...
    usable_doors = self._filter_backtrack_doors(doors, direction_hint)
    nearest_door = self._get_nearest(usable_doors, direct=direction_hint) if
direction_hint else None
    if nearest_door is None and direction_hint:
        raise LookupError("no door matches route hint")
    if nearest_door is None and usable_doors:
        nearest_door = self._get_nearest(usable_doors)
    if nearest_door:
        self._move_to_door(nearest_door["xywh"])
    return
```

这一段的作用

当前房间清完后，程序会按路线方向去筛门、找门、朝门移动。

执行逻辑

先拿方向提示，再过滤掉回头门，然后优先找该方向上的最近门。没有严格匹配门时，会触发后面的兜底逻辑，而不是贸然走错门。

它是干什么的

这是房间级路线真正落实到当前画面行为的关键桥梁。

片段 20：异常处理和最终兜底移动

函数 `run()` 结尾

```
except LookupError:
    pass
except Exception as exc:
    logger.exception("game loop error: {}", exc)
    self._release_cached_action()
    return

fallback_direction = self._current_door_direction()
if fallback_direction:
    if self._should_block_backtrack_direction(fallback_direction):
        self._release_cached_action()
        return
    self._fallback_route_move(fallback_direction)
    return

self._release_cached_action()
```

这一段的作用

当前面所有主要分支都没成功时，程序会尝试做一个安全的后备动作；如果连后备方向都没有，就停住。

执行逻辑

没有清晰目标时，程序不会硬闯，而是优先做可控的兜底移动，或者干脆释放按键等待下一帧。

它是干什么的

这是动作系统最后一道安全阀，防止“没想清楚就乱动”。

第七章：minimap_route_demo.py

这个文件负责什么： 这是一个单独测试导航链路的脚本，不是主刷图入口。它用来验证“截图 + 检测 + 小地图导航”这一层是否正常。

片段 1：初始化和截图测试

文件主体

```
def main() -> None:
    width = 1067
    height = 600
    detector = Detector("")
    navigator = MiniMapNavigator(map_name="generic")

    while True:
        started = time.time()
        img = pyautogui.screenshot(region=(0, 0, width, height))
        frame_rgb = np.array(img)
        annotated, objects = detector.detect(frame_rgb)
        frame_bgr = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR)
```

这一段的作用

用固定区域截图来快速测试检测和导航，不需要进入完整刷图主循环。

执行逻辑

初始化检测器和导航器后，循环截取屏幕指定区域，并把结果送进检测和导航模块。

它是干什么的

这是一个方便开发者调试导航链路的测试脚本。

片段 2：打印导航结果并显示调试窗口

文件后半段

```
snapshot = navigator.build_route_snapshot(frame_bgr, objects or [])
print(
    json.dumps(
        {
            "current_room": snapshot.current_room,
            "boss_room": snapshot.boss_room,
            "query_room": snapshot.query_room,
            "elite_room": snapshot.elite_room,
            "next_room_direction": snapshot.next_room_direction,
            "selected_door_center": snapshot.selected_door_center,
            "cost_ms": round((time.time() - started) * 1000, 1),
        },
        ensure_ascii=False,
    )
)
```

这一段的作用

把导航结果打印成 JSON，同时显示大图检测结果和小地图窗口。

执行逻辑

这样开发者可以非常直观看到“导航层此刻认为自己在哪、目标房在哪、下一步往哪走”。

它是干什么的

这是一个专门验证导航是否准的体检工具。

附录：为什么 `olaplug` 不放进主线逐段讲解

不是遗漏，而是为了抓主线。 `olaplug` 目录主要是一个功能非常大的 DLL 封装层，里面有窗口绑定、输入、图色、OCR、YOLO、内存接口等大量通用函数。但当前这套刷图主流程并没有真正实例化 `OLAPlugServer`，也没有调用 `BindWindow()`。

当前真实执行链

你现在最该记住的主线

win32gui 找窗口

- > pyautogui 截图
- > detector.py 做 YOLO 检测
- > minimap_nav.py 做小地图识别并调用 A 星
- > game.py 用 pydirectinput 发方向键 / 攻击键 / 拾取键

这一段的作用

提醒你精力优先放在当前真正运行的刷图主链上。

执行逻辑

如果你一开始就钻进 `olaplug`，很容易被大量通用接口淹没，反而看不清这套刷图代码现在到底是怎么工作的。

它是干什么的

帮助你聚焦真正的主线，而不是被备用工具箱带偏。

文档基于当前工作区 `D:\yolo\yolo11\dnf` 源码重新整理，修复了旧版本中的乱码片段和空白过多问题。